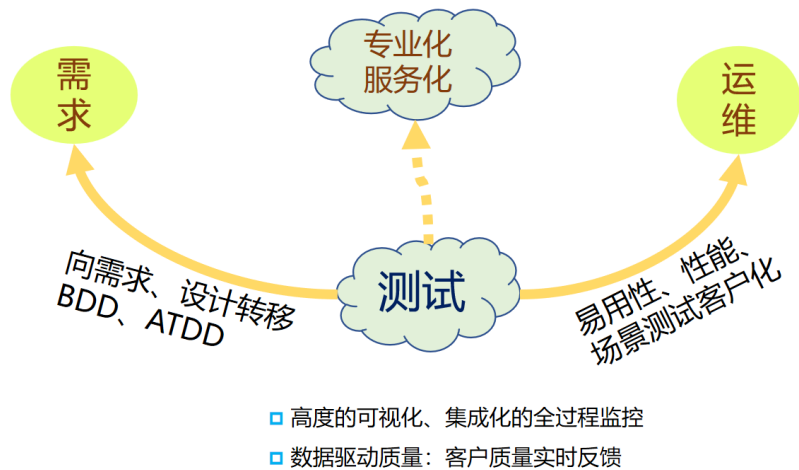


第四章 软件测试流程和规范

- 软件测试贯穿全生命周期

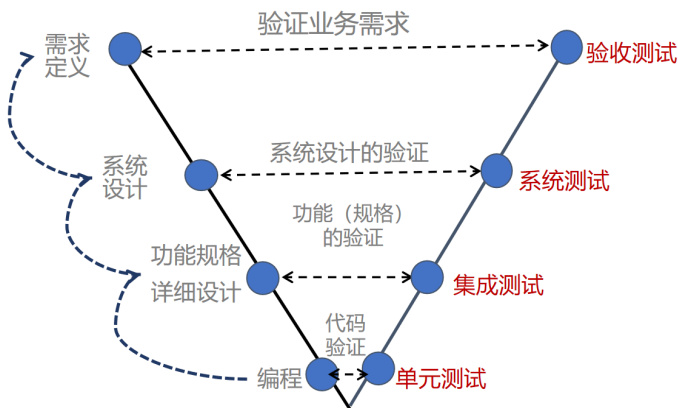


- 测试左移
 - 开发人员做更多的测试，还要参与需求评审、设计评审等。
 - 测试计划、测试设计可以在更早时候开始。
 - ATDD, TDD
- 测试右移
 - 是在线测试（Test in Production, TiP），包括在线性能监控与分析、A/B测试和日志分析等，可以和现在流行的DevOp联系起来。

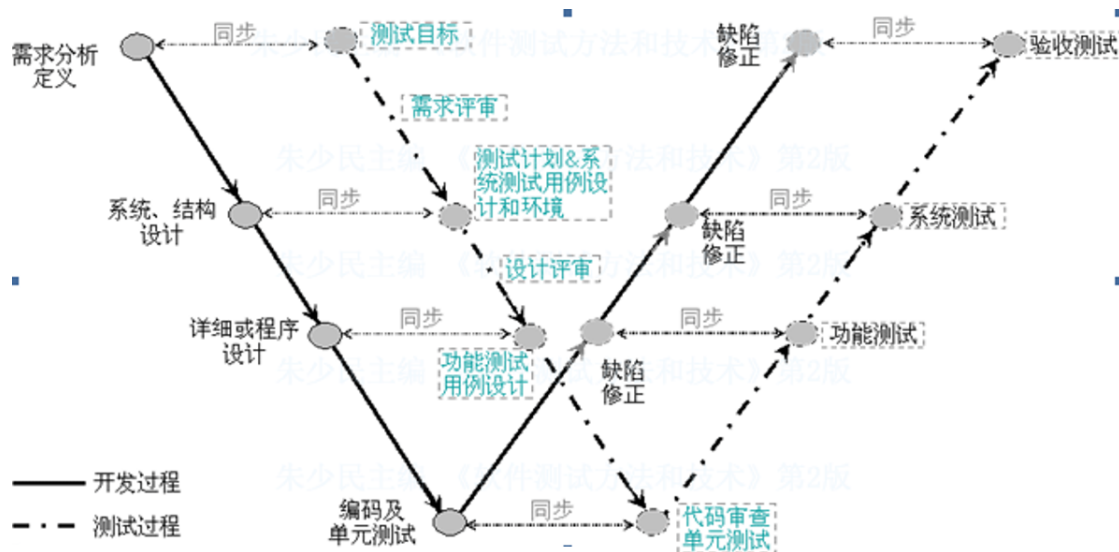
- 4.1 传统的软件测试过程



系统测试的层次



- 4.1.1 W模型

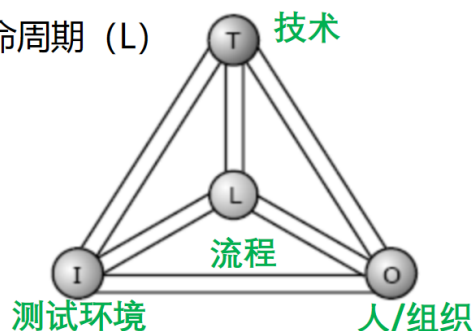


- (1) 测试过程和开发过程同时开始，同时结束，两者保持同步关系。
- (2) 测试过程是对开发过程中阶段性成功和最终的产品进行验证的过程，两者相互依赖。前期测试过程更多地依赖于开发的过程；后期开发的过程更多地依赖于测试的过程。
- (3) 测试过程中的工作重点和开发工作的重点可能是不一样的。在资源管理、风险管理等多方面都存在差异。

• 4.1.2 TMap

- 一个基于风险的测试方法
- TMap (Test Management Approach, 测试管理方法) 是一种结构化的、基于风险策略的测试方法体系，目的能更早地发现缺陷，以最小的成本、有效地、彻底地完成测试任务，以减少软件发布后的支持成本。
- TMap所定义的测试生命周期由计划和控制、准备、说明、执行和完成等阶段组成。
- O、I、T三大基石

- 与软件开发生命周期一致的测试活动生命周期 (L)
- 坚实的组织融合 (O)
- 正确的基础设施和工具 (I)
- 可用的技术 (T)



• 4.2 敏捷测试过程

• 敏捷宣言背后的原则

- ① 尽早和持续地交付有价值的软件来满足客户
- ② 欢迎需求变更——即使是在项目开发后期。要善于利用需求变更，帮助客户获得竞争优势
- ③ 要不断交付可用的软件，周期从几周到几个月不等，且越短越好
- ④ 项目过程中，业务人员与开发人员必须在一起工作

- ⑤ 要善于激励项目人员，给他们以所需要的环境和支持，**并相信**他们能够完成任务
- ⑦ 无论是团队内还是团队间，最有效的**沟通方法是面对面的交谈**
- ⑧ **可用的软件**是衡量进度的主要指标
- ⑨ 敏捷过程提倡**可持续的开发**。项目方、开发人员和用户应该能够保持长期稳定的开发速度
- ⑨ 对技术的**精益求精**、对设计的不断完善将提升敏捷性
- ⑩ **简单**——尽最大可能减少不必要的工作——一门艺术
- ⑪ 最佳的架构、需求和设计出自于**自组织的团队**
- ⑫ 团队要**定期反省**如何能够做到更有效，并相应地调整团队的行为

● 敏捷测试特征

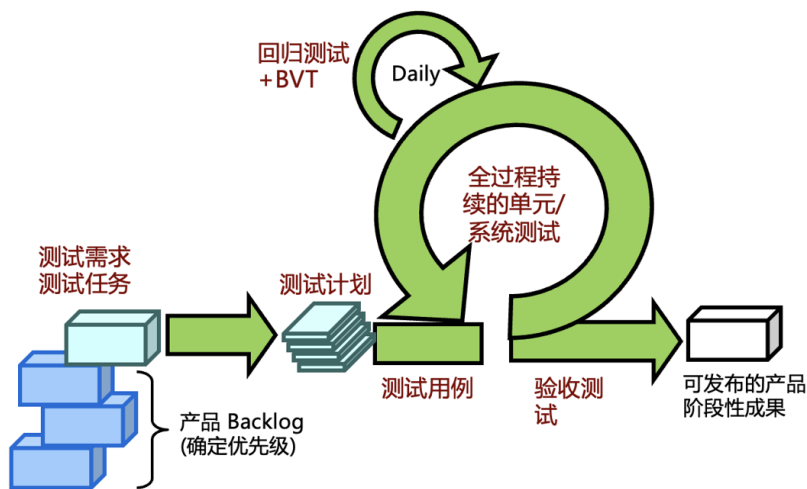
- ① 尽早和持续地开展测试
- ② 能及时完成对软件质量全面评估
- ③ 软件本身是测试研究和分析最主要的对象
- ④ 在满足所要求的质量，测试进行得越快越好
- ⑤ 测试人员必须和项目干系人保持密切协作
- ⑥ 对测试人员足够信任和尊重
- ⑦ 测试计划、设计和执行力求简单
- ⑧ 对测试技术精益求精
- ⑨ 不断反思，持续优化测试设计

● 传统测试和敏捷测试的区别

传统测试	敏捷测试
❑ 重流程 ❑ 阶段性明确 ❑ 测试团队独立性 ❑ 测试人员职责细化 ❑ 关注需求/测试文档 ❑ 分析/文档能力 ❑ 严谨、规范 ❑ 缺陷预防 ❑ 测试过程改进	❑ 以人为本，强调个人技能 ❑ TDD、ATDD ❑ 持续测试 ❑ 侧重产品的测试 ❑ ET、TA ❑ 整个团队对测试负责 ❑ 拥抱变化 ❑ 批判性思维能力 ❑ 自我学习能力

PK

- 进一步理解传统测试和敏捷测试的不同点和共同点？
- 敏捷Scrum测试流程



• 基于会话的测试管理（SBTM）

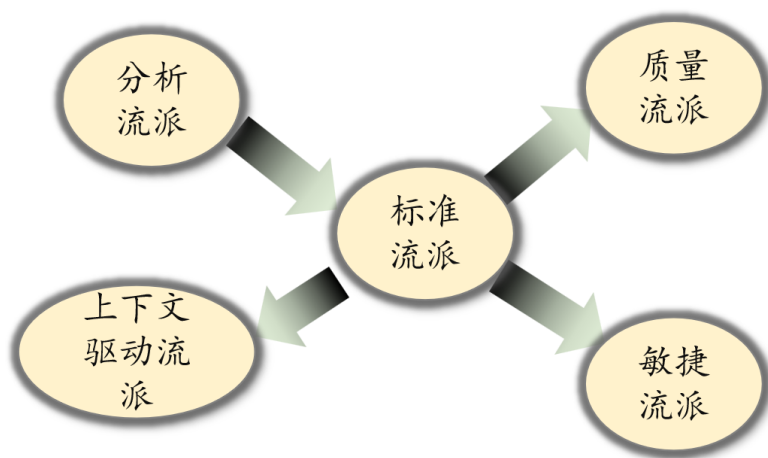
- **Session** (会话) 是一段不受打扰的测试时间（通常是90分钟），是测试管理的最小单元。
- 每个session关联一个特定的、目标明确的测试任务（**mission**）
- **Charter**（章程，即测试指导）：对每个session如何执行进行简要的描述，相当于每个session需要一个简要的计划（提纲）
- 一系列Session相互支持，有机地组合在一起，**周密地测试了整个产品**。
- **A session sheet**（测试报告）：相当于测试报告，供第三方（如测试经理、ScrumMaster等）进行检查的材料。它最好能被工具扫描、解析和合成。
- **Debriefing**（听取口头报告）：口头汇报，更准确地说，是测试人和其lead/Manager之间的对话。

Charter: 进一步明确测试范围和策略

- Charter 清晰指导Session执行任务（Mission）：要测什么、怎么测试（强调策略，不是详细测试步骤）、寻找什么样的缺陷、哪些产品质量风险等
- 可以看做一个或一组用例（User Case）的体现，相对用例（**use case**）的测试思路

• 4.3 软件测试流派

• 五大软件测试流派



- ① **分析流派**: 认为测试是严格的和技术性的, 在学术界有许多支持者
- ② **标准流派**: 将测试视为衡量进度的一种方法, 强调成本和可重复的标准
- ③ **质量流派**: 强调过程规范性, 监督开发人员并充当质量的看门人
- ④ **上下文驱动流派**: 强调人的价值, 寻找涉众关心的bug
- ⑤ **敏捷流派**: 强调自动化测试, 使用测试来快速验证开发是完整的

上下文驱动测试方法

CDT: Context-driven Testing

- ① 任何实践活动的价值依赖于它所处的上下文
- ② 在某个上下文中只有好的实践, 没有最佳实践
- ③ 一起工作的人, 才是项目的最重要组成部分
- ④ 项目的发展往往难以预料
- ⑤ 产品是问题的解决方案, 如果问题没得到解决, 产品是无用的
- ⑥ 好的软件测试是一个富有挑战性的智力过程。
- ⑦ 只有通过判断和技能, 并在整个项目过程中协同练习它们, 我们才能在正确的时间做正确的事, 以有效地测试我们的产品

• 4.4 软件测试改进

• TMMi

过程能力描述了遵循一个软件测试过程可能达到的预期结果的范围。TMMi的建立, 得益于以下3点:

- 充分吸收、CMM/CMMi的精华;
- 基于历史演化的测试过程;
- 业界的最佳实践。

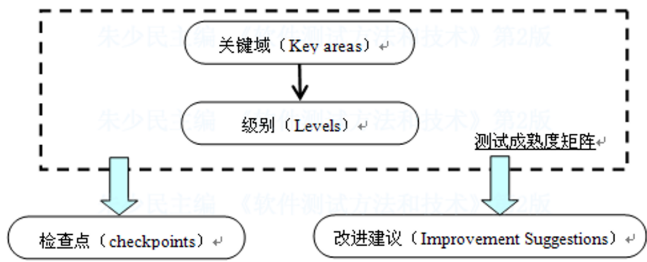
- 5个级别的一系列测试能力成熟度的定义, 每个级别的组成包括到期目标、到期子目标活动、任务和职责等。
- 一套评价模型, 包括一个成熟度问卷、评估程序和团队选拔培训指南。

• 5个级别的成熟度: 课本P88

- 1.初始级
- 2.阶段定义级
- 3.集成级
- 4.管理和度量级
- 5.优化级

• TPI NEXT

TPI（Test Process Improvement）是基于连续性表示法的测试过程改进的参考模型，是在软件控制、测试知识以及过往经验的基础上开发出来的



- 为了能客观地决定各个关键域的级别，TPI模型提供了一种度量工具——检查点。每个级别都有若干个检查点，测试过程只有在满足了这些检查点的要求之后，才意味着它达到了特定的级别
- 检查点帮助我们发现测试过程中的问题，而建议会帮助我们解决问题，最终改进测试过程。建议不仅包含对如何达到下个级别的指导，而且还包括一些具体的操作技巧、注意事项等。

TPI 检查点和建议

- 为了能客观地决定各个关键域的级别，TPI模型提供了一种度量工具——检查点。每个级别都有若干个检查点，测试过程只有在满足了这些检查点的要求之后，才意味着它达到了特定的级别
- 检查点帮助我们发现测试过程中的问题，而建议会帮助我们解决问题，最终改进测试过程。建议不仅包含对如何达到下个级别的指导，而且还包括一些具体的操作技巧、注意事项等。

TPI 级别

- 为了了解过程在每个关键域所处的状态，即对关键域的评估结果，通过级别是来体现。模型提供了4个级别，由A到D，A是最低级。根据测试过程的可视性改善、测试效率的提高、或成本的降低以及质量的提高，级别会有所上升。

TPI成熟度矩阵

总体等级 关键域	可控的					有效的					不断优化的			
	0	1	2	3	4	5	6	7	8	9	10	11	12	13
测试策略	A	C					B				C		D	
生命周期模型	A			B										
介入时间		A					B				C		D	
估计和计划			A								B			
测试规格技术	A		B											
静态测试技术				A			B							
度量					A			B				C		D
测试自动化			A					B			C			
测试环境			A					B						C
办公环境			A											
承诺与动力	A					B						C		
测试功能与培训			A				B				C			
方法的范围				A							B			C
沟通			A		B								C	
报告	A			B			C					D		
缺陷管理	A					B		C						

TPI 20个关键域

- | | |
|-----------|-------------|
| 1. 测试策略 | 11. 承诺与动力 |
| 2. 生命周期模型 | 12. 测试功能与培训 |
| 3. 介入时间 | 13. 方法的范围 |
| 4. 估计和计划 | 14. 沟通 |
| 5. 测试规格技术 | 15. 报告 |
| 6. 静态测试技术 | 16. 缺陷管理 |
| 7. 度量 | 17. 测试件管理 |
| 8. 测试自动化 | 18. 测试过程管理 |
| 9. 测试环境 | 19. 评估 |
| 10. 办公环境 | 20. 底层测试 |

- 本章小结

- 传统软件测试流程：W模型、TMap Next
- 敏捷测试流程：敏捷宣言、敏捷开发12项原则、TDD、SBTM
- 五大测试流派：重点理解上下文驱动测试流派
- 软件测试改进：重点TMMi和TPI next
- 软件测试标准与规范